Real Resale Value

JAYCUS GOH SI PENG, A0311404Y
GRACE TAN YU JIN, A0311421B
LETITIA LIEW ANQI, A0311376J
NG WEE SENG, A0311393L

Data Sources

data.gov.sg

HDB Transaction Data

month	town	flat_type	block	street_name	storey_range	floor_area_sqm	flat_model	lease_commence_date	remaining_lease	resale_price
2017-01	ANG MO KIO	2 ROOM	406	ANG MO KIO AVE 10	10 TO 12	44.0	Improved	1979	61 years 04 months	232000.0
2017-01	ANG MO KIO	3 ROOM	108	ANG MO KIO AVE 4	01 TO 03	67.0	New Generation	1978	60 years 07 months	250000.0
2017-01	ANG MO KIO	3 ROOM	602	ANG MO KIO AVE 5	01 TO 03	67.0	New Generation	1980	62 years 05 months	262000.0
2017-01	ANG MO KIO	3 ROOM	465	ANG MO KIO AVE 10	04 TO 06	68.0	New Generation	1980	62 years 01 month	265000.0
2017-01	ANG MO KIO	3 ROOM	601	ANG MO KIO AVE 5	01 TO 03	67.0	New Generation	1980	62 years 05 months	265000.0

Resale Price Index (RPI)

quarter	index
2017-Q1	133.9
2017-Q2	133.7
2017-Q3	132.8
2017-Q4	132.6
2018-Q1	131.6

Geospatial Data

name	category	lat	long
The Star Vista	mall	1.308002	103.788382
Bencoolen Underground Mall	mall	1.299224	103.850755
Katong V	mall	1.303133	103.903231
The Poiz Centre	mall	1.331436	103.868571
Clarke Quay Central	mall	1.288999	103.846180

CSV


```
{
  "type": "FeatureCollection",
  "name": "MRT_EXITS",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
        "coordinates": [103.83363541399386, 1.3640371821656152]
      },
      "properties": {
        "OBJECTID": 17885,
        "STATION_NA": "BRIGHT HILL MRT STATION",
        "EXIT_CODE": "Exit 1",
        "INC_CRC": "CFB9350F44E8991A",
        "FMEL_UPD_D": "20251202172807"
      }
    },
```


GeoJSON

Geospatial Data

GeoPandas

https://www.onemap.gov.sg/api/common/elastic/search?searchVal=406%20ANG%20MO%20KIO%20AVE%2010&returnGeom=Y&getAddrDetails=Y&pageNum=1


```
{
  "found": 1,
  "totalNumPages": 1,
  "pageNum": 1,
  "results": [
    {
      "SEARCHVAL": "406 ANG MO KIO AVENUE 10 SINGAPORE 560406",
      "BLK_NO": "406",
      "ROAD_NAME": "ANG MO KIO AVENUE 10",
      "BUILDING": "NIL",
      "ADDRESS": "406 ANG MO KIO AVENUE 10 SINGAPORE 560406",
      "POSTAL": "560406",
      "X": "30288.2346631354",
      "Y": "38229.0674628187",
      "LATITUDE": "1.36200453938712",
      "LONGITUDE": "103.85387991040 7"
    }
  ]
}
```


GPS Coordinates

OneMap Singapore

GPS to metres

4326 - Global, spheroid, degree-based

3414 - Projected, Singapore localised, 2D, metre-based

```
loc = gpd.GeoDataFrame(
    df_geo,
    geometry=gpd.points_from_xy(
        df_geo["long"],
        df_geo["lat"],
    ),
    crs="EPSG:4326",
)

loc_m = loc.to_crs(epsg=3414)
loc_m["x_m"] = loc_m.geometry.x
loc_m["y_m"] = loc_m.geometry.y
```

block	street_name	Location	lat	long	geometry	x_m	y_m
406	ANG MO KIO AVE 10	406 ANG MO KIO AVE 10	1.36200453938712	103.853879910407	POINT (30288.235 38229.067)	30288.234663	38229.067463
108	ANG MO KIO AVE 4	108 ANG MO KIO AVE 4	1.37096635222625	103.838201940326	POINT (28543.459 39220.01)	28543.458747	39220.009892
602	ANG MO KIO AVE 5	602 ANG MO KIO AVE 5	1.38070883044887	103.835368226602	POINT (28228.1 40297.283)	28228.099954	40297.283149
465	ANG MO KIO AVE 10	465 ANG MO KIO AVE 10	1.3662010408294	103.857200967235	POINT (30657.825 38693.099)	30657.824693	38693.098657
601	ANG MO KIO AVE 5	601 ANG MO KIO AVE 5	1.38104134784496	103.835131744823	POINT (28201.782 40334.051)	28201.782487	40334.051212

EPSG:3414

Coordinate Reference System (CRS) maintained by the European Petroleum Survey Group (EPSG)

CRS

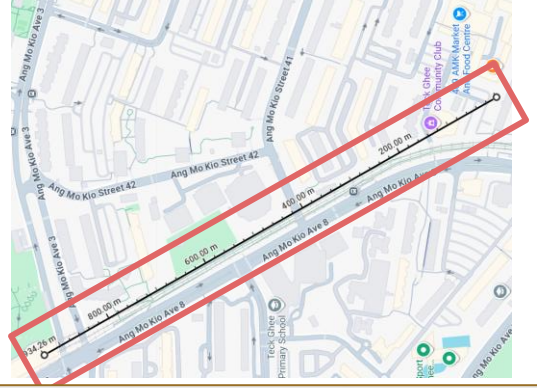
Using *sjoin_nearest()* to find shortest distance from each location to its nearest MRT exit

Nearest Distances

```
nearest_mrt = gpd.sjoin_nearest(
    loc_m,
    mrt_m[["STATION_NA", "EXIT_CODE", "geometry"]],
    how="left",
    distance_col="dist_to_mrt_m",
)
nearest_mrt[[
        "Location",
        "STATION_NA",
        "EXIT_CODE",
        "dist_to_mrt_m",
]].sort_values("dist_to_mrt_m", ascending=True)
```

Data Preprocessing

Transformed Data Set



date	floor_area_sqm	storey_mid	remaining_lease_months	town	flat_type	flat_model	dist_to_mrt_m	dist_to_mall_m	dist_to_school_m	dist_to_hawker_m	dist_to_spf_m	RPI	resale_price
2017-01-01	44.0	11.0	736	ANG MO KIO	2 ROOM	Improved	934.249034	1027.883036	227.902618	172.410689	700.649811	133.9	232000.0
2017-01-01	67.0	2.0	727	ANG MO KIO	3 ROOM	New Generation	160.176577	869.965513	241.572335	408.448982	136.555811	133.9	250000.0
2017-01-01	67.0	2.0	749	ANG MO KIO	3 ROOM	New Generation	390.204129	1526.601124	775.445626	586.167266	1108.541396	133.9	262000.0
2017-01-01	68.0	5.0	745	ANG MO KIO	3 ROOM	New Generation	885.083182	892.872255	518.558573	244.836572	741.171455	133.9	265000.0
2017-01-01	67.0	2.0	749	ANG MO KIO	3 ROOM	New Generation	376.976459	1570.330140	781.266765	611.702356	1149.265548	133.9	265000.0

Nearest Distances (metres)

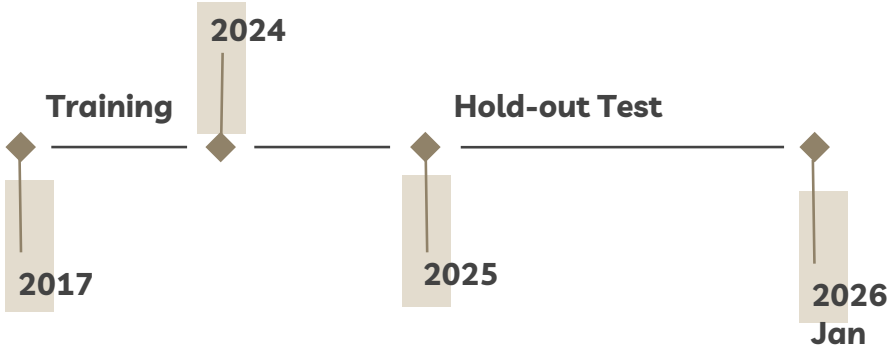
```
df["date"] = pd.to_datetime(df["month"])
df["quarter"] = df["date"].dt.to_period('Q').astype(str)
df["quarter"] = df["quarter"].astype(str).str.replace('Q', '-Q')
df["quarter"]
```

```
lease_parts = df["remaining_lease"].str.extract(
    r"(?:(\d+)\s+years?)?\s*(?:(\d+)\s+months?)?"
)
lease_parts = lease_parts.fillna(0).astype(int)
df["remaining_lease_months"] = lease_parts[0] * 12 +
lease_parts[1]
```

```
tmp = df["storey_range"].str.extract(r"(\d+)\s*TO\s*(\d+)")
df["storey_low"] = pd.to_numeric(tmp[0], errors="coerce")
df["storey_high"] = pd.to_numeric(tmp[1], errors="coerce")
df["storey_mid"] = (df["storey_low"] + df["storey_high"]) / 2
```

Time-Based Train-Test Split

```python
data = df[["date"] + feature_cols + [target]].copy()

cutoff = pd.Timestamp("2024-12-31")

train = data[data["date"] <= cutoff]
test  = data[data["date"] > cutoff]
```



- ✓ Reduces information leakages
- ✓ Reflects a more realistic forecasting setting
- ✓ Historical transactions used to predict future resale prices

Modelling Pipeline

```python
# Preprocessing pipeline: keep numeric features as it is and one-hot encode categorical features
preprocess = ColumnTransformer(
    transformers=[
        ("num", "passthrough", num_features),
        ("cat", OneHotEncoder(handle_unknown="ignore"), cat_features)
    ],
    remainder="drop"
)
```

One-hot encoding on categorical

```python
# Create a function to run time-series cross-validation for a given model
def run_timeseries_cv(model, preprocess, X_train, y_train, n_splits=5, n_jobs=-1, log_y=False):

    # Wrap model so that fitting is done on log(y) and predictions are transformed back to the original target scale
    if log_y:
        model = TransformedTargetRegressor(regressor=model, func=np.log, inverse_func=np.exp)

    # Build full pipeline with preprocessing and model
    pipe = Pipeline([
        ("prep", preprocess),
        ("model", model)
    ])

    # Time-series split to preserve temporal order
    tscv = TimeSeriesSplit(n_splits=n_splits)

    # Evaluation metrics for cross-validation
    scoring = {
        "mae": "neg_mean_absolute_error",
        "rmse": "neg_root_mean_squared_error",
        "r2": "r2"
    }
```

Pipeline

5-fold split

Scoring metrics

Modelling Pipeline

Cross-validation variable

```python
# Perform cross-validation
cv = cross_validate(
    pipe,
    X_train,
    y_train,
    cv=tscv,
    scoring=scoring,
    n_jobs=n_jobs
)
```

```python
# Convert negative error scores back to positive values
mae_scores = -cv["test_mae"]
rmse_scores = -cv["test_rmse"]
r2_scores = cv["test_r2"]
```

Return evaluation table

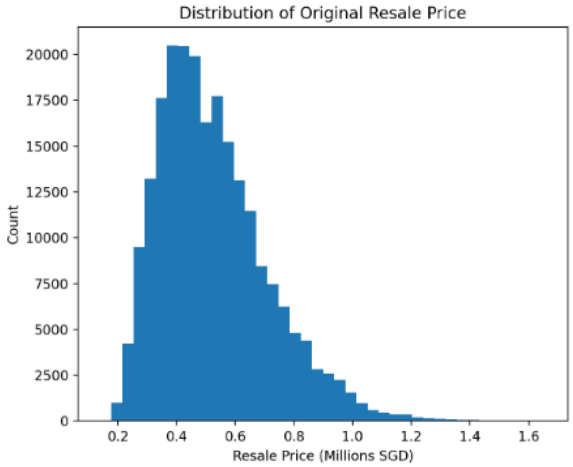
```python
# Create dataframe of fold-by-fold results
cv_df = pd.DataFrame({
    "Fold": range(1, len(mae_scores)+1),
    "MAE": mae_scores,
    "RMSE": rmse_scores,
    "R2": r2_scores
})
```

```python
# Add row of mean performance across folds
mean_row = pd.DataFrame({
    "Fold": ["Mean"],
    "MAE": [mae_scores.mean()],
    "RMSE": [rmse_scores.mean()],
    "R2": [r2_scores.mean()]
})

# Append mean row to cv fold dataframe
cv_df = pd.concat([cv_df, mean_row], ignore_index=True)

# Round values for cleaner display
cv_df = cv_df.round({"MAE":0, "RMSE":0, "R2":4})

return cv_df
```

Linear Regression

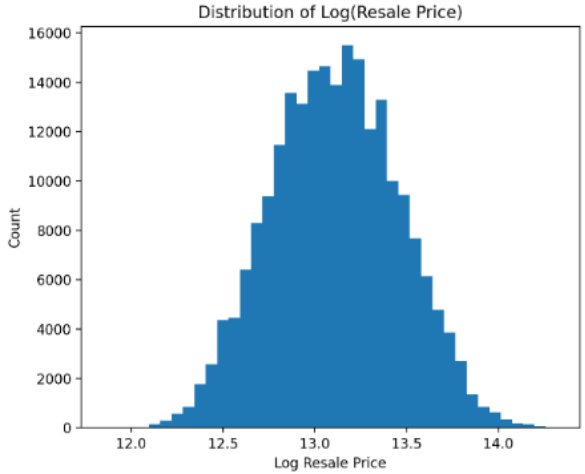
Linear Regression

- Baseline model
- Simple, interpretable
- Assumes a linear relationship between predictors and resale price
- Model_lr = LinearRegression()



Log-Linear Regression

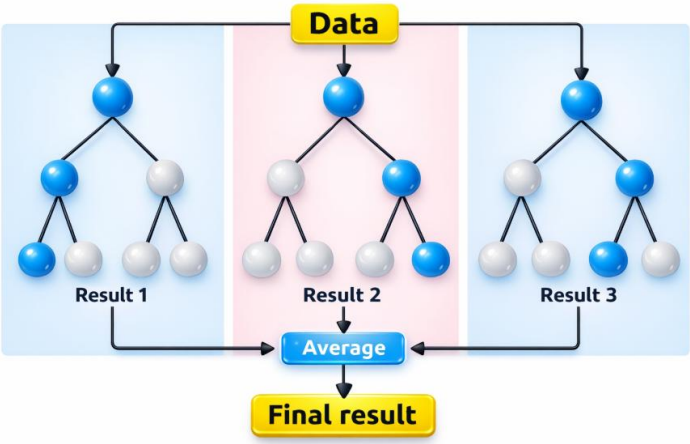
- Resale price was right-skewed
- Log transformation helps reduce skewness
- Tested to improve model fit
- Skewness: 0.958 → 0.035



Random Forest

Random Forest Regressor

- Ensemble model that combines many decision trees
- Captures non-linear relationships and complex patterns
- Compared 10 trees vs 300 trees
- Log-transformed tested for comparison




```
model_rf_300 = RandomForestRegressor(n_estimators=300, random_state=42, n_jobs=-1)
```

number of decision trees in the forest

fixes the random seed

tells the model to use all available CPU cores

Model Evaluation – Cross Validation

Mean Absolute Error (MAE): Measures the average absolute error in dollar terms

Root Mean Squared Error (RMSE): Penalizes larger errors more heavily

R-squared: Measures overall explanatory power

Lower MAE/RMSE and higher R-squared indicate stronger performance

Linear Regression

```
lr_cv = run_timeseries_cv(LinearRegression(), preprocess, X_train, y_train)
lr_cv
```

Fold	MAE	RMSE	R2
1	40802.0	53405.0	0.8803
2	43048.0	57246.0	0.8700
3	42673.0	56127.0	0.8860
4	44712.0	60040.0	0.8793
5	49666.0	67832.0	0.8678
Mean	44180.0	58930.0	0.8767

Linear Regression - log(y)

```
lr_log_cv = run_timeseries_cv(LinearRegression(), preprocess, X_train, y_train, log_y=True)
lr_log_cv
```

Fold	MAE	RMSE	R2
1	34986.0	48217.0	0.9024
2	35180.0	48911.0	0.9051
3	37553.0	51035.0	0.9058
4	38274.0	51390.0	0.9116
5	42434.0	69514.0	0.8611
Mean	37685.0	53813.0	0.8972

Model Evaluation – Cross Validation

Random Forest Regressor

```
model_rf = RandomForestRegressor(n_estimators=10, random_state=42, n_jobs=-1)
rf_cv = run_timeseries_cv(model_rf, preprocess, X_train, y_train)
rf_cv
```

Fold	MAE	RMSE	R2
1	28672.0	41778.0	0.9267
2	31380.0	44333.0	0.9220
3	50349.0	64693.0	0.8486
4	40103.0	55015.0	0.8987
5	43707.0	60744.0	0.8940
Mean	38842.0	53313.0	0.8980

300-tree Random Forest Regressor

```
model_rf_300 = RandomForestRegressor(n_estimators=300, random_state=42, n_jobs=-1)
rf_300_cv = run_timeseries_cv(model_rf_300, preprocess, X_train, y_train)
rf_300_cv
```

Fold	MAE	RMSE	R2
1	26892.0	38828.0	0.9367
2	30335.0	43082.0	0.9264
3	49398.0	63071.0	0.8561
4	38980.0	53513.0	0.9041
5	42586.0	59380.0	0.8987
Mean	37638.0	51575.0	0.9044

Random Forest Regressor - log(y)

```
model_rf = RandomForestRegressor(n_estimators=10, random_state=42, n_jobs=-1)
rf_log_cv = run_timeseries_cv(model_rf, preprocess, X_train, y_train, log_y=True)
rf_log_cv
```

Fold	MAE	RMSE	R2
1	29001.0	42969.0	0.9225
2	32834.0	47335.0	0.9111
3	51696.0	66756.0	0.8388
4	41275.0	56764.0	0.8921
5	44497.0	61638.0	0.8908
Mean	39860.0	55092.0	0.8911

300-tree Random Forest Regressor - log(y)

```
model_rf_300 = RandomForestRegressor(n_estimators=300, random_state=42, n_jobs=-1)
rf_300_log_cv = run_timeseries_cv(model_rf_300, preprocess, X_train, y_train, log_y=True)
rf_300_log_cv
```

Fold	MAE	RMSE	R2
1	27215.0	40116.0	0.9324
2	31356.0	45280.0	0.9187
3	51145.0	65546.0	0.8446
4	40337.0	55348.0	0.8974
5	43630.0	60654.0	0.8943
Mean	38737.0	53389.0	0.8975

Model Evaluation

Final Model Comparison:

Linear Regression

```
final_model_lr = LinearRegression()

final_pipe_lr = Pipeline(steps=[
    ("prep", preprocess),
    ("model", final_model_lr)
])

final_pipe_lr.fit(X_train, y_train)

pred_lr = final_pipe_lr.predict(X_test)

mae_lr  = mean_absolute_error(y_test, pred_lr)
rmse_lr = mean_squared_error(y_test, pred_lr, squared=False)
r2_lr   = r2_score(y_test, pred_lr)
```

```
Test MAE:  54,428
Test RMSE: 75,857
Test R2:   0.8618
```

```
final_model_log_lr = TransformedTargetRegressor(
    regressor=LinearRegression(),
    func=np.log,
    inverse_func=np.exp
)
```

```
Log-LR Test MAE:  44,952
Log-LR Test RMSE: 60,124
Log-LR Test R2:   0.9132
```

300-trees Random Forest

```
final_model_rf = Rando
    n_estimators=300,
    random_state=42,
    n_jobs=-1
)
```

```
RF Test MAE:  34,114
RF Test RMSE: 49,791
RF Test R2:   0.9405
```

```
final_model_log_rf = TransformedTargetRegressor(
    regressor=RandomForestRegressor(
        n_estimators=300,
        random_state=42,
        n_jobs=-1
    ),
    func=np.log,
    inverse_func=np.exp
)
```

```
Log-RF Test MAE:  35,165
Log-RF Test RMSE: 51,822
Log-RF Test R2:   0.9355
```

Final Model Selection

```
final_test_df = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Log Linear Regression",
        "Random Forest",
        "Log Random Forest"
    ],
    "MAE": [mae_lr, mae_log_lr, mae_rf, mae_log_rf],
    "RMSE": [rmse_lr, rmse_log_lr, rmse_rf, rmse_log_rf],
    "R2": [r2_lr, r2_log_lr, r2_rf, r2_log_rf]
}).round({"MAE": 0, "RMSE": 0, "R2": 4})

final_test_df
```

Model	MAE	RMSE	R2
Linear Regression	54428.0	75857.0	0.8618
Log Linear Regression	44952.0	60124.0	0.9132
Random Forest	34114.0	49791.0	0.9405
Log Random Forest	35165.0	51822.0	0.9355

Key Price Drivers

```python
# Extract the fitted Random Forest model and transformed feature names
rf = final_pipe_rf.named_steps["model"]
feature_names = final_pipe_rf.named_steps["prep"].get_feature_names_out()

# Create and sort feature importances in descending order
imp = pd.Series(
    rf.feature_importances_,
    index=feature_names
).sort_values(ascending=False)
```

Feature	Importance	Importance_pct
floor_area_sqm	0.399	39.9
RPI	0.148	14.8
storey_mid	0.090	9.0
remaining_lease_months	0.069	6.9
dist_to_mrt_m	0.027	2.7

THANK YOU!